**Instituto Politecnico Nacional**

**ESCOM “ESCUELA SUPERIOR DE CÓMPUTO”**

*ANÁLISIS DE ALGORITMOS*

*DISEÑO DE SOLUCIONES DyV*

PROFE: Edgardo Adrián Franco Martínez

ALUMMNO: Rojas Alvarado Luis Enrique

GRUPO: 3CM4

DivideConquer1.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long int ull;

int main(){
    int n;                          //Se declara la variable en donde.
    cin >> n;                       //se define como tamaño del arreglo.
    vector<ull> A(n);               //se hace un vector A con el tamaño n.
    for(int i = 0; i < n; ++i){     //ciclo que agrega el tamaño al vector.
        cin >> A[i];                //se introduce número por número.
    }
    ull ans = A[0];                 //arreglo que tendrá la suma más grande (resultado).
    ull tmp_max = A[0];             //arreglo que tendrá la máxima suma hasta el momento.
    for(int i = 1; i < n; ++i){     //ciclo en el que se hacen las sub-sumas.
        tmp_max = max(tmp_max + A[i], A[i]);    //se suman los elementos contiguos.
        ans = max(tmp_max, ans);                //se saca el número máximo.
    }
    cout << ans;                    //arroja la suma máxima.
    return 0;
}
```

El orden de complejidad es O(n) puesto que solo consta de dos ciclos for.

## II. Cúmulo.

omegaUp | Arena | Concursos | Problemas | Ranking | Escuelas | Blog | Preguntas | Wicho

### Cumulo

| | | | |
|---|---|---|---|
| Puntos | 16.49 | Límite de memoria | 32 MiB |
| Límite de tiempo (caso) | 1s | Límite de tiempo (total) | 1m0s |
| Límite de entrada (bytes) | 10 KiB | | |

#### Cumulo

#### Descripción

Te encuentras con un mapa del cúmulo de estrellas R136. En el mapa, cada estrella aparece como un punto ubicada en un plano cartesiano. Te asalta de pronto una pregunta, ¿cuál será la distancia mínima entre dos estrellas en el mapa?

#### Entrada

La primera línea tendrá un entero $2 <= n <= 50000$ que indica la cantidad de estrellas en el mapa. Las siguientes n líneas tendrán las coordenadas de las estrellas, dadas por dos reales X y Y. En todos los casos, $0 <= X, Y <= 40000$.

#### Salida

La distancia mínima entre dos estrellas, expresada con un número real con tres cifras después del punto decimal. (La distancia se calcula como la raíz cuadrada de la suma de los cuadrados de las diferencias en X y Y)

#### Ejemplos

| Entrada | Salida | Descripción |
|---|---|---|
| 3<br>0.0 0.0<br>2.0 0.0<br>3.0 3.0 | 2.000 | Las estrellas más cercanas son las primeras dos. |
| 5<br>10.8 142.5<br>20.2 14.05<br>112.4 0.2<br>24.2 17.05<br>0.0 512.3 | 5.000 | Las estrellas más cercanas son la segunda y la cuarta. |

## Límites

- $2 <= n <= 50000$
- $0 <= X, Y <= 40000$ (X,Y reales)

*Fuente: Alain*
Problema subido por: Alain_Acevedo
Reportar contenido inapropiado en este problema.

Envíos

| Enviado | GUID | Estatus | Porcentaje | Lenguaje | Memoria | Tiempo | Detalle |
|---|---|---|---|---|---|---|---|
| 2019-04-29 18:51:43 | 1c92e4db | Respuesta correcta | 100.00% | cpp | 3.98 MB | 1.21 s | |

Se tiene que ordenar los puntos según la coordenada x y trazar una línea vertical que divida a los dos puntos, el par más cercano puede estar en cualquiera de esos puntos si estuviesen en $P_1$ o en $P_2$ se analiza recursivamente, subdividiendo en más problemas o si está un punto en cada zona tendremos 3 distancias por si está en $P_1$ y en $P_2$ o si está uno en $P_1$ y el otro en $P_2$ o si estuvieran en $P_1$ o $P_2$ y MinimaDistancia al menor de la distancia 1 y distancia 2, para resolver el último caso solo se necesita ver en los puntos cuya coordenada x esté en Centro-MinimaDistancia y Centro+MinimaDistancia.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <algorithm>
#include <iostream>
#include <string.h>
typedef long long int lli;

using namespace std;
struct Punto{                                          // Estructura de un Punto.
   double x;
   double y;
 };
void Busca(struct Punto *,int);
int Ordenax(const void *,const void *);
double Distancia(struct Punto,struct Punto);
double MinimaDistancia=1e10;                           // Mínima distancia.


int main()
{
   int a,n;                                            //Algunas variables
   double x,y;
   cin>>n;                                             //Obten n
   struct Punto p[n];                                  //Declara el arreglo de puntos

   for(int i=0;i<n;i++)                                // Obten los datos
   {
      scanf(" %lf %lf",&x,&y);
```

```c
      p[i].x=x;
      p[i].y=y;
   }

   Busca(p,n);                                   //Busca la minima distancia

   printf("%.3lf\n",MinimaDistancia);

   return(0);
 }

void Busca(struct Punto *p,int n)
{
   double d;
   int i,j,a,b;

   if(n<=1)                                      // Si no hay pares de puntos:
      return;                                    // salir.

   qsort(p,n,sizeof(struct Punto),Ordenax);      // Ordenar los puntos por la coordenada x.

   Busca(p,n/2);                                 // Mirar en el subconjunto de la izquierda.

   Busca(p+n/2,(n+1)/2);                         // Mirar en el subconjunto de la derecha.
                                                 //Busca del lado derecho e izquiero los puntos mas cercanos
   for(i=n/2; (i>0) && (p[n/2].x-p[i].x<MinimaDistancia); i--); // Hallar los límites del conjunto central.

   for(j=n/2; j<n-1 && p[j].x-p[n/2].x<MinimaDistancia; j++);   //Búsqueda en el conjunto central.
   for( a = i ; a <= j ;a ++)
```

```c
      for( b = a + 1 ; b <= j ; b++)
         if((d=Distancia(p[a],p[b]))<MinimaDistancia)          //Si encuentras una distancia mínima actualizala
         {
            MinimaDistancia=d;
         }
 }


int Ordenax(const void *a,const void *b){
   return(((*(struct Punto *)a).x<(*(struct Punto *)b).x)?-1:1);     //Manera en que se comparan los puntos x
 }

double Distancia(struct Punto a,struct Punto b){                  // Función que calcula la distancia entre dos puntos.
   return(sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y)));
 }
```

Puesto que se mandan 2 búsquedas y cada una con la mitad del problema, podemos escribir que: $T(n) = 2T(\frac{n}{2})+2n$ y resolviendo por teorema maestro: a = 2; b = 2; F(n) = 2n; => $n^{\log_2 2} = n^1 = n$ => $T(n) = \theta(n \log n)$.

## III. INVCNT - Inversion Count

Sphere online judge — PROBLEMAS · ESTADO · Rangos · DISCUTIR · Los concursos · registrarse

Problemas / clásico / Cuenta de inversión — Estado · Clasificación

### INVCNT - Cuenta de inversión

# teoría-gráfica # teoría -numérica # ruta más corta # ordenando #mascaras

Sea A [0 ... n - 1] una matriz de n enteros positivos distintos. Si i <j y A [i]> A [j], entonces el par (i, j) se llama inversión de A. Dado n y una matriz A, su tarea es encontrar el número de inversiones de A.

#### Entrada

La primera línea contiene t, el número de cajas de prueba seguidas de un espacio en blanco. Cada una de las pruebas t comienza con un número n (n <= 200000). Luego siguen n + 1 líneas. En la línea i se da un número A [i - 1] (A [i - 1] <= 10 ^ 7). La línea (n + 1) th es un espacio en blanco.

#### Salida

Para cada salida de prueba, una línea da el número de inversiones de A.

#### Ejemplo

```
Entrada:
2
```

Enviar solución!

La nueva generación de herramientas de gestión de proyectos está aquí y es visual.
ANUNCIOS A TRAVÉS DE CARBONO

| | |
|---|---|
| Añadido por: | Androide paranoico |
| Fecha: | 2010-03-06 |
| Límite de tiempo: | 3.599s |
| Límite de fuente: | 50000B |
| Limite de memoria: | 1536MB |
| Racimo: | Cubo (Intel G860) |
| Idiomas: | Todos excepto: PERL6 |

```
Entrada:
2

3
3
1
2

5
2
3
8
6
1


Salida:
2
5
```

Ordenamos el arreglo por Merge Sort y contamos las inversiones realizadas. Si ya se sabe el número de inversiones se suman en número de inversiones y de la combinación tendremos el número de inversiones en el arreglo.

Si en cualquier paso del merge: A[ i ] > A[ j ] entonces hay centro – i inversiones puesto que los arreglos izquierdo y derecho se ordenan, entonces todos los elementos restantes en subcampo izquierdo son mayores a A[ j ].

Sphere online judge — PROBLEMS · STATUS · RANKS · DISCUSS · CONTESTS · PROFILE

judge status

< Previous 1 2 3 4 5 Next >

| ID | DATE | USER | PROBLEM | RESULT | TIME | MEM | LANG |
|---|---|---|---|---|---|---|---|
| 23700832 | 2019-04-30 03:57:37 | wicho | Inversion Count | accepted edit ideone it | 0.12 | 19M | CPP |

slack

```
1   #include <algorithm>
2   #include <iostream>
3   #include <vector>
4   #include <string.h>
5   typedef long long int lli;
6
7   using namespace std;
8   lli  MergeSortInversiones(lli Numeros[], lli ArrayTemporal[], lli Izquierda, lli Derecha);
9   lli Merge(lli Numeros[], lli ArrayTemporal[], lli Izquierda, lli Centro, lli Derecha);
10
11
12  /* Crea un array temporal y llama a merge sort*/
13  lli MergeSort(lli Numeros[], lli n)
14  {
15      lli *ArrayTemporal = (lli *)malloc(sizeof(lli)*n);
16      return MergeSortInversiones(Numeros, ArrayTemporal, 0, n - 1);
17  }
18  //Realiza el merge sort y regresa el número de inversiones
19  lli MergeSortInversiones(lli Numeros[], lli ArrayTemporal[], lli Izquierda, lli Derecha)
20  {
21    lli Centro, NumInversiones = 0;
22    if (Derecha > Izquierda)
23    {
24
25      Centro = (Derecha + Izquierda)/2;                                    //Divide el arreglo en 2 partes
26
27      NumInversiones  = MergeSortInversiones(Numeros, ArrayTemporal, Izquierda, Centro);
28      NumInversiones += MergeSortInversiones(Numeros, ArrayTemporal, Centro+1, Derecha);
29      NumInversiones += Merge(Numeros, ArrayTemporal, Izquierda, Centro+1, Derecha);        //Combina las 2 partes
30    }
31    return NumInversiones;
32  }
33
34  ///Combina 2 arreglos ordenados y regresa el número de inversiones de los arreglos
35  lli Merge(lli Numeros[], lli ArrayTemporal[], lli Izquierda, lli Centro, lli Derecha)
36  {
37    lli i, j, k;
38    lli NumInversiones = 0;
39
40    i = Izquierda;                                          //Indice del subarray izquierdo
41    j = Centro;                                              //Indice del subarray derecho
42    k = Izquierda;                                           //Indice del subarray combinado resultante
43    while ((i <= Centro - 1) && (j <= Derecha))
44    {
45      if (Numeros[i] <= Numeros[j]){
46        ArrayTemporal[k++] = Numeros[i++];
47      }
48      else{
49        ArrayTemporal[k++] = Numeros[j++];
50        NumInversiones += (Centro - i);                        //Si no esta ordenado hay Centro-i inversiones
51      }
52    }
53
54
55    while (i <= Centro - 1)                                    //Copia los elementos restantes del subarray Izquierdo al array temporal
56      ArrayTemporal[k++] = Numeros[i++];
57
58    while (j <= Derecha)                                       //Copia los elementos restantes del subarray Derecho al array temporal
59      ArrayTemporal[k++] = Numeros[j++];
60
61    for (i=Izquierda; i <= Derecha; i++)                       //Copia los elementos mezclados al array original
62      Numeros[i] = ArrayTemporal[i];
63
64    return NumInversiones;
65  }
66
67  int main(){
68    lli t=0;                                                   //Obtenemos el número de casos
69    cin>>t;                                                    //Obtenemos el número de casos
70    char resultado[100000]="";                                 //Donde guardaremos la respuesta
71
72    for (lli i = 0; i < t; ++i){
73      lli n=0;
74      cin>>n;
75      lli Numeros[n];
76          for(lli i=0; i<n;i++)
77          {
78              cin>>Numeros[i];                                 // Se lee el numero
79          }
80          sprintf(resultado+strlen(resultado),"%lld\n",MergeSort(Numeros, n));
81          n=0;
82    }
83    printf("%s\n",resultado );
84    return 0;
85  }
```

Puesto que es un merge y dividimos el problema en izquierda y derecha, tenemos la siguiente ecuación recurrente:

$T(n) = 2T(\frac{n}{2}) + n$

$a = 2; \quad b = 2; \quad f(n) = n$

Por teorema maestro tenemos que:

$n^{\log_2 2} = n^1 = n$

Por lo que la cota para éste algoritmo es:

$T(n) = \theta(n \log n)$.